

11.3 Aider

Aider inventor and developer Paul Gauthier describes his work somewhat succinctly as “Aider is AI pair programming in your terminal.” In fact, Aider lacks a web or graphical user interface (GUI) interface. However, the rest of the description is an understatement from our point of view. Microsoft describes GitHub Copilot very similarly as “Your AI pair programmer.”

With regard to functionality, however, Copilot and Aider are worlds apart! Aider expects relatively abstract instructions, automatically creates the required files or makes changes to existing project files, and finally asks whether you want to apply the changes as a Git commit. As we’ve already indicated in the introduction to this chapter, Aider acts much more independently than GitHub Copilot, Continue, or other plug-ins that are integrated into the editor or the development environment.

11.3.1 Installation

Aider is open-source software based on Python and Git—prerequisites that are almost certainly already fulfilled on your computer. To avoid package conflicts with other projects, it’s best to install Aider in a virtual environment. We carried out our tests on macOS with Python 3.12.

```
mkdir aider    # Installation for macOS/Linux
cd aider
python3 -m venv .
source bin/activate
pip3 install aider-chat
```

On Windows, you need to use the `python` and `pip` commands instead of `python3` and `pip3`. You also need to activate the environment using

Scripts/Activate.ps1 instead of the source command. Updates can be carried out as follows:

```
pip3 install aider-chat --upgrade
```

For Aider to work, the program must have access to a language model. Assuming suitable hardware, a (strong!) local model is sufficient. We tested Aider with two commercial language models, GPT-4o from OpenAI and Claude Sonnet 3.5 from Anthropic. Sonnet 3.5 delivered noticeably better results in our tests.

By the time you read this book, there will be newer language models so we can't really tell which one will be best for Aider. But take a look at the following page, which summarizes which language model works how well with Aider: <https://aider.chat/docs/leaderboards>.

Now you need to define an environment variable for your API key, preferably in `.bashrc` or `.zshrc` on Linux or macOS. Alternatively, you can also save the key(s) in `.env` in the Aider environment directory.

To receive an API key, you must create an account with OpenAI or Anthropic and enter your contact and credit card details. You buy credits there in advance, which are then used up by API queries. About \$10 should be sufficient for initial tests, so you don't have to invest a fortune to try out Aider.

```
# in ~/.bashrc oder ~/.zshrc or aider/.env
export OPENAI_API_KEY='sk-xxxxx'
export ANTHROPIC_API_KEY='sk-yyyyy'
```

If one of the two variables is defined, Aider automatically recognizes the desired model. Once both variables have been defined, Aider opts for the Sonnet model from Anthropic. If required, you can explicitly select the language model with the `--4o` or `--sonnet` options.

11.3.2 Start

Aider is a terminal tool, but that doesn't mean you have to make do without your editor. A good way to use it is to open a project directory in your favorite editor (we used Visual Studio Code [VS Code]). For a new project, this is initially empty. Then, open a terminal window in the editor, and activate the Aider virtual environment. Make the project directory the active directory.

```
cd ~/aider          # change to Aider directory
source bin/activate  # activate virtual environment
cd ~/myproject      # change to the project
directory
```

Now start the AI tool using `aider`. If there is no Git repository in the local directory, Aider recommends creating one. All you have to do is confirm the corresponding query.

```
aider
```

```
Aider v0.50.1
Models: claude-3-5-sonnet-20240620 with diff edit format,
       weak model claude-3-haiku-20240307
Use /help <question> for help, run "aider --help"
       to see cmd line args
```

11.3.3 Aider Commands

For new projects, you can start directly by giving the tool instructions, for example, which task your new program should perform or which programming language you want to use. Aider creates the required new files itself. To run and test the program, you can either use the editor, a development environment, or simply a second terminal window.

Of course, you can also use Aider to edit an existing project with many files. Aider commands such as `/command` play an important role in this type of work (see [Table 11.2](#)). You can find a complete reference at <https://aider.chat/docs/usage/commands.html>.

While the created files are added directly to the context for new projects, you must explicitly add the files to be edited to the context for existing projects via `/add` or `/read` or remove them again by using `/drop`. The difference between `/add` and `/read` is that Aider only makes changes in the `/add` files during further prompts, while it considers the `/read` file only as a reference. Files passed with `/read` can, for example, contain the code of a class whose methods you want to use, documentation, or schema files.

A major problem with all language models is that their information status ends with the date on which the training material was created. This is a huge limitation, especially if you use modern programming languages or libraries.

Aider gets around this problem with a simple concept: you can use `/web` to transfer the address of a website. Then, Aider downloads this page, converts the text found there into Markdown format, and inserts it into the context.

Command	Meaning
<code>/add file</code>	Adding a code file to the context
<code>/ask</code>	Asking questions about the code (no changes)
<code>/clear</code>	Deleting the chat history (files remain in context)
<code>/diff</code>	Showing changes from the last commit
<code>/drop file</code>	Removing a code file from the context
<code>/exit</code> or <code>/quit</code>	Exiting Aider
<code>/git cmd</code>	Executing a Git command (e.g., <code>/git status</code>)
<code>/help myquestion</code>	Asking the language model a general question

Command	Meaning
<code>/help /cmd</code>	Displaying help for the specified command
<code>/lint</code>	Checking the syntax of current files, fixing errors
<code>/ls</code>	Showing files in the context
<code>/map</code>	Showing a repository overview
<code>/read</code>	Adding a file to a context (read-only)
<code>/run cmd</code>	Executing a shell command (e.g., <code>/run cd subdir</code>)
<code>/undo</code>	Revoking the last commit
<code>/web url</code>	Reading a file and adding it to the context

Table 11.2 Important Aider Commands

Less Is More

It's not a good idea to add as many or even all files of a large project to the context! This is also true for the number of pages transferred from an on-line documentation using `/web`.

We've already mentioned it several times in this book: although modern language models have pleasingly large context windows, they struggle to separate the essential from the nonessential. Too much contextual information often unbalances the language model. The choice of what is important or not so important is up to you!

11.3.4 Multiline Prompts

As soon as you press `Enter`, Aider considers the entry to be complete and starts responding. Multiline prompts can be entered via `Alt` + `Enter`. However, experience shows that there is a high risk of forgetting `Alt`. The

second approach therefore seems more helpful to us: start the entry with a curly bracket, and then press `Enter`. You can then enter as many additional lines as you like and press `Enter` before ending your prompt with another curly bracket.

```
> {  
> this is a long question  
> about the meaning of life  
> }
```



It often happens that predefined conventions for the code must be adhered to in a project. In this case, the Aider documentation recommends saving the rules as ordinary text instructions in a Markdown file in the project directory. You can add this file to the context with `/read` or pass it as a parameter when starting `aider`:

```
aider code-conventions.md
```

11.3.6 Aider in the Web Browser

Aider sees itself as a terminal tool. The program can also be operated in the web browser. To do this, you need to start the program with the additional `--browser` option. A new tab with the Aider web interface will then appear in your web browser.

The Aider manual describes this function as experimental, and we can confirm this impression: the interface is visually appealing, but despite asking questions in English, Aider formulated the answers in German (the default language set in our web browser), made changes to the code despite `/ask`, didn't understand commands such as `/undo`, and so on.